Interactive Problems

- Gaurish Baliga

What are Interactive Problems?

- Unlike conventional problems where you get an input and you have to print an output for it, in Interactive Problems, your code interacts with the Online Judge's code.

- Online Judge gives input -> Your code prints an output which acts as input for the Online Judge -> Online Judge prints an output which acts as input for your code and the interaction goes on…

- Mostly this revolves around making your code ask the right questions to the Online Judge and getting to a correct answer.

What are Interactive Problems?

- Sounds scary? All you have to do is read the problem clearly, understand the constraints, number of queries allowed and then come up with an algorithm that gets you the answer quickly without exceeding those constraints.

- Most Important: Flush the output after every print statement.
    - What is flushing?
        - endl vs \n

endl

'\n'

flushing

10,000

??

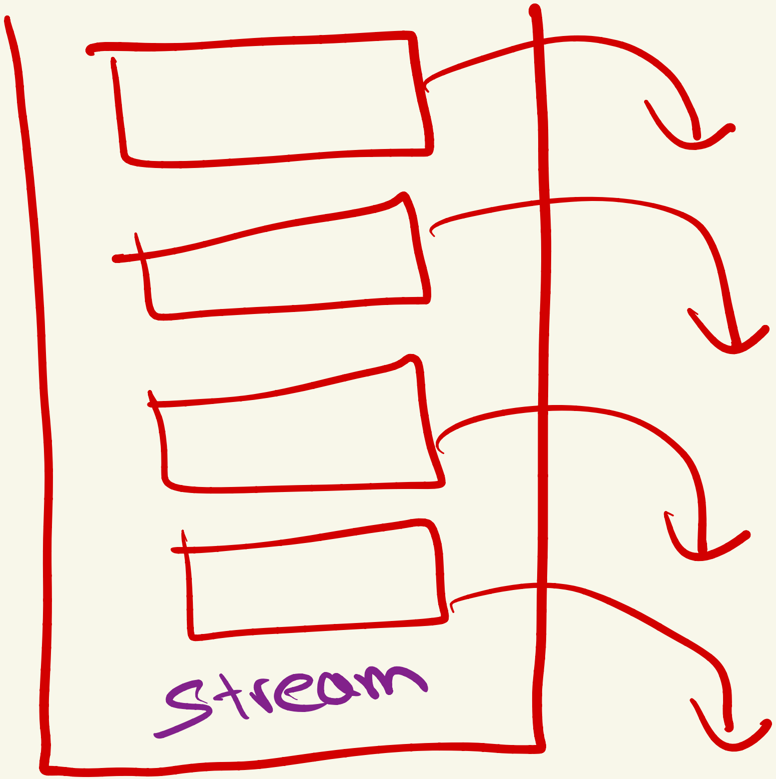
↳ Yes / No

→ Yes

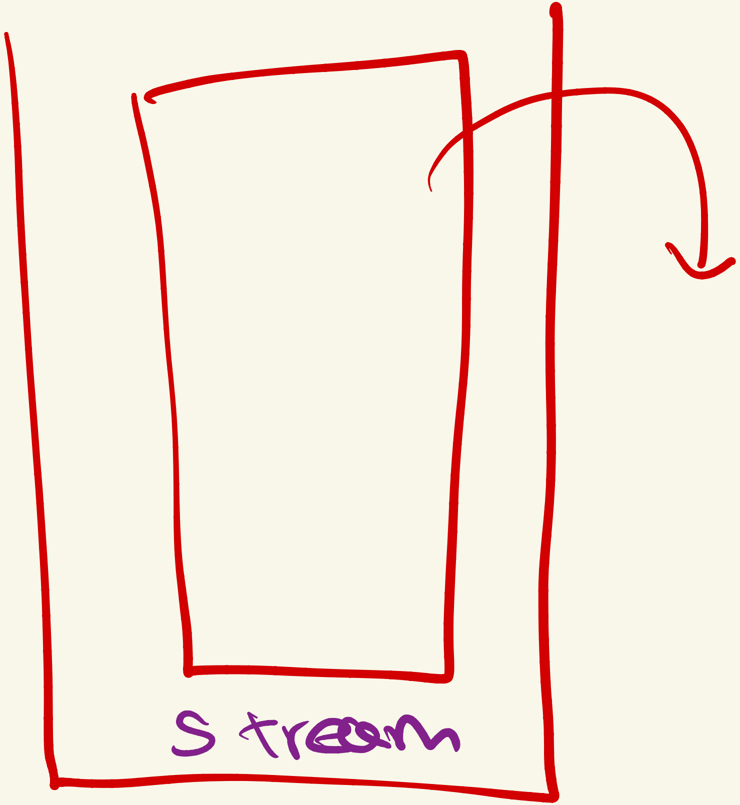
→ No

→ Yes

→ n



endl



\n

Problem

There is a hidden number. You can guess a number and the computer will tell you if the number is equal to the hidden number, greater or smaller. Find the number quickly. When found, print **! X** as your answer.

Answer should be found in less than or equal to 25 queries, 0<=n<=1e5

Example Query: If the hidden number is 10

? 2

< The computer replies with ‘<’ because clearly 2 < 10.

? 10

= The computer replies with ‘=’ because 10 = 10

! 10

Hidden Number = 10

Your query	? 2	2 is less than 10
Computer's Answer	<	
Your query	? 8	8 is still less than 10
Computer's Answer	<	
Your query	? 11	11 is greater than 10
Computer's Answer	>	
Your query	? 10	Found it
Computer's Answer	=	
Printing the Final Answer	! 10	

Problem

There is a hidden array of integers, you need to find any index of the majority element or report that no such element exists. You can ask for any index and the computer will tell you if that index contains the majority element or not.

Majority element is the one which occurs more than floor(n/2) times. Answer should be found in less than or equal to 100 queries, 0<=n<=1e9

Example Query: Hidden array [10, 11, 10]

? 2

0 [2nd index contains 11 and it is not the majority element]

? 1

1 [1st index contains 10 and it is the majority element]

Hidden Array = [9, 10, 9, 9, 9, 10, 11, 10, 9, 9]

Your query	? 2	10 is not majority element
Computer's Answer	0	
Your query	? 8	11 is not majority element
Computer's Answer	0	
Your query	? 1	9 is majority element
Computer's Answer	1	
Printing the Final Answer	! 1	

No pattern of how the majority elm can accur

5, 5, 5, 3, 3

3, 3, 5, 5, 5

3, 5, 5, 5, 3

→ mx occurs $\geq n/2$ times

index i $\xrightarrow{\text{is mx}} \geq \frac{1}{2}$

index i $\xrightarrow{\text{not mx}} \leq \frac{1}{2}$

n

$$\underbrace{i, j} \qquad \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} = 0.25$$

$$\underbrace{i, j, k} = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{8} = \frac{1}{2^{q}}$$

$q = 30$

100

$\times$

$\frac{1}{2^{30}} \to \frac{1}{10^9} =$

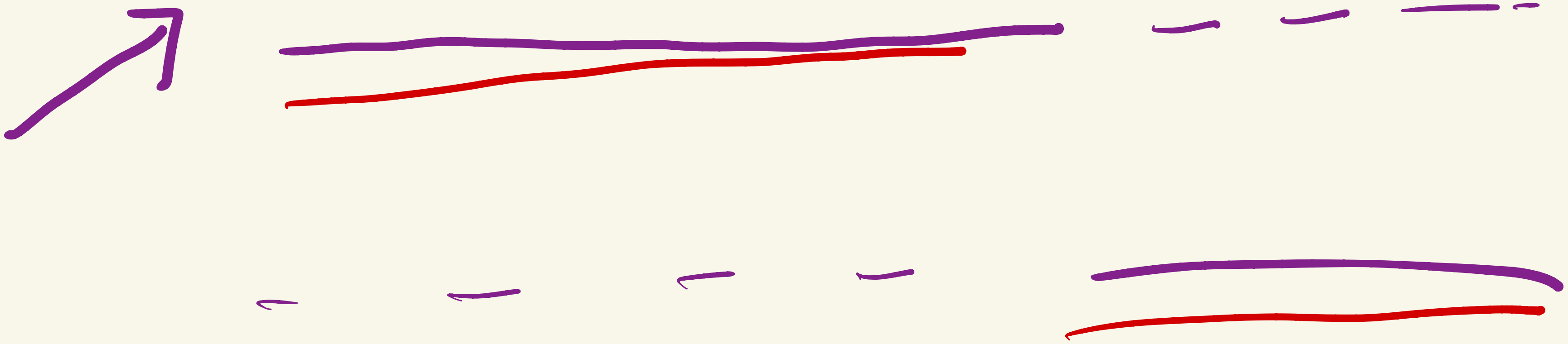
$$\boxed{\frac{1}{2^{100}}}$$

$$\boxed{\frac{1}{10^{30}}}$$

Probabilistic

99.9999 ... %

→ choose 100 random indices & ask for those 100 random indices



$\underline{0}$ __ __

$$\boxed{\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2}}$$

$$\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} \left(mx \cdot nm \cdot nm \right) + nm \cdot mx \cdot nm$$

$$mx \cdot mx \cdot nm$$

How to test Interactive Problems?

Building a Custom Interactor which acts as the Online Judge and your code interacts with the custom interactor. Let's look at the code for this…

Important Pointers

Remember these 2 concepts and you will solve almost all Interactive Problems:

- Binary Search / Ternary Search
- Randomization - Non deterministic, but once you solve a lot of problems using this, you will realise the power of this thing. Really easy to code and gives amazing results.

Side Note: Practice a lot of Constructive algo problems to improve your thinking for interactive problems.

Ternary search